



BACKEND ARCHITECTURE & IMPLEMENTATION MANUAL

OQQQ 后端开发文档

统一承载 Hermes、Claude Code、Codex 与 DeepSeek Harness 的本地多 Agent 协作核心：消息、路由、工作区、能力、审批、组队、摘要与项目归档。

文档版本

v0.1 / 2026-10-07

读者

后端、桌面端、适配器与测试工程师

实施基线

桌面优先 · 本地优先 · 契约先行

配套

OQQQ 前端开发对接文档 v0.3

核心域归 OQQQ，运行时通过适配器替换

OQQQ 不是某个 Agent Web UI 的换皮。后端必须稳定拥有用户可见的会话、消息、联系人、工作区、审批和项目生命周期；外部引擎仅提供“思考与执行”能力。

1

统一领域模型

前端只看 OQQQ 的 Conversation、Message、Run、Agent、Capability 和 Archive，不感知供应商事件格式。

2

命令与事件分离

REST 接收有结果的命令；WebSocket 传输增量事件。所有命令可幂等重试，所有事件可按序恢复。

3

审批高于运行时

写文件、执行命令、外部发送和凭据使用由 OQQQ Core 最终裁决，不能依赖引擎默认模式。

本版本冻结的决策

主题	决定	直接结果
传输	REST v1 + WebSocket v1	前端只实现一个 Data Client
存储	SQLite + 会话独立工作区	单机易部署，数据可迁移
执行	每次 Agent 调用创建 Run	支持取消、审批、重试与审计
路由	单聊隐式；群聊显式 @	避免群成员无边界抢答
摘要	派生快照，不覆盖原消息	可回溯、可重算
适配	SDK/服务优先，CLI 降级	减少解析 stdout 的脆弱性

非目标：首版不嵌入 dsh Web UI，不实现跨设备云同步，不让任意 Agent 自动安装第三方代码，也不把密钥写入消息、日志或项目档案。

从领域边界读到可执行契约

正文按“架构 - 领域 - 协议 - 适配 - 运维”组织。前端联调优先阅读 REST 与 WebSocket；运行时开发优先阅读 Agent 适配器与 dsh；项目能力开发阅读 PM、摘要和档案。

目录

02 系统架构

03 请求与事件链路

04 模块边界

05 消息与 @ 路由

06 会话与工作区

07 PM 动态组队

08 摘要与项目档案

09 数据模型

10-11 REST API

12 WebSocket

13 Agent 适配器

14-15 引擎与 dsh

16 安全与可靠性

17 测试与观测

18 阶段计划与验收

阅读约定： 示例 ID、路径与数值仅用于说明协议形状。真实实现以 packages/contracts 中的 OpenAPI 与 JSON Schema 为唯一可执行契约。

六层结构隔离产品语义与供应商差异

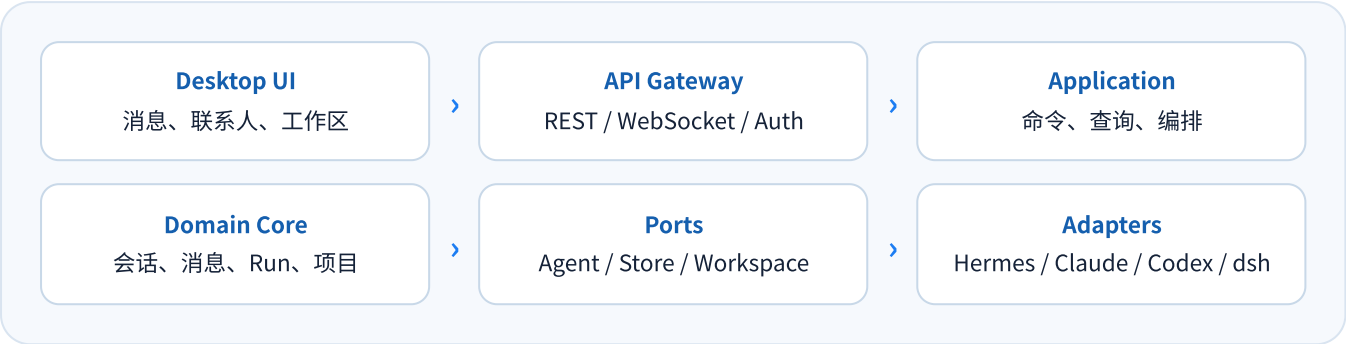


图 1：前端不直接调用 CLI；所有供应商输入先在适配器层归一化。

控制平面

- **Agent Registry**：发现、安装、升级、配置与健康状态。
- **Capability Registry**：插件、技能、连接器及其兼容性。
- **Policy Engine**：权限、审批、工作区边界与外部发送规则。
- **Project Orchestrator**：PM 组队、任务分派、项目完结和恢复。

数据平面

- **Message Bus**：消息持久化、事件发布、订阅与恢复。
- **Run Coordinator**：启动、流式、取消、超时和错误归一化。
- **Workspace Service**：目录绑定、文件版本、变更通知和冲突。
- **Archive Service**：可移交快照、校验、恢复计划与导出。

建议运行拓扑

进程	职责	故障影响	恢复策略
oqqq-desktop	界面与本地资源协议	UI 不可用	重启后从 cursor 追赶
oqqq-core	API、领域、SQLite	停止收发	WAL 恢复；Run 标记 interrupted
agent-sidecar	单一引擎适配器进程	对应 Agent 离线	指数退避，限次重启
dsh-sdk	工具、技能、连接器	能力不可执行	健康探针；one-shot 回退

边界原则：适配器崩溃不能破坏 OQQQ Core；Core 崩溃后不能把“已发送但未确认”的命令重复执行。通过 run_id、idempotency_key 与事件序列共同保证。

消息先落库，再路由，再启动 Run

发送一条群消息

- 1 客户端提交命令。 `POST /v1/conversations/{id}/messages`，携带客户端生成的 `clientId` 和 `Idempotency-Key`。
- 2 Core 事务落库。校验成员、工作区与附件后，写入 Message 与 OutboxEvent，返回 `202 Accepted`。
- 3 总线发布。Outbox worker 发出 `message.appended`；WebSocket 客户端立即显示服务端正式消息。
- 4 @ 路由解析。Router 以结构化 mention ID 为准，生成 0..N 个 Invocation；正文里的“@名字”只用于显示，不用于权限判断。
- 5 Run Coordinator 排队。每个目标 Agent 对应独立 Run；同一 Agent 同一会话默认串行，不同 Agent 可并行。
- 6 适配器流式归一化。供应商事件被映射为 `message.delta`、`tool.*`、`run.state_changed` 等 OQQQ 事件。
- 7 完成与提交。最终 Assistant Message 单次写入；增量只用于显示。Run 记录使用量、时长、错误和退出原因。

关键时序约束

先 ACK，后执行

HTTP 只确认消息已接受，不等待模型完成。执行结果由 WebSocket 持续返回，避免长请求与代理超时。

Delta 非事实源

断线时可丢弃未提交 delta。恢复后以最终 Message 和 Run 状态为准，不从 UI 拼接结果回写。

审批会暂停 Run

进入 `waiting_approval` 后禁止继续执行对应工具。批准或拒绝必须引用 `approval_id`。

取消是协作协议

Core 发出 `cancel`，适配器尽力中断；若超时则终止子进程并把 Run 标为 `cancelled_forced`。

禁止：收到 WebSocket delta 后直接当成已完成消息保存；以 Agent 显示名做路由；在 CLI stdout 中混入 OQQQ 自己的控制协议。

模块按所有权划分，而不是按页面划分

模块	拥有的数据	公开能力	不负责
identity	User、DeviceSession	本地身份、连接鉴权	供应商账号
conversation	Conversation、Member	创建、成员、公告、草稿	Agent 执行
messaging	Message、Attachment、Reaction	写入、分页、引用、撤回	路由策略
routing	RouteDecision、Mention	@解析、目标选择、队列	模型调用
runs	Run、RunEvent、Approval	启动、暂停、取消、重试	引擎细节
agents	Engine、AgentInstance	发现、安装、健康、配置	联系人 UI
capabilities	Plugin、Skill、Connector	目录、安装、绑定、权限	市场页面
workspace	Workspace、FileVersion	树、读取、写入、变更	任意路径访问
projects	Project、TeamPlan、Task	组队、阶段、完结、恢复	通用聊天
memory	Summary、ContextSnapshot	触发、压缩、引用范围	删除原始消息
archives	Archive、Manifest	生成、校验、导出、恢复	保存凭据

推荐代码布局

```
apps/core/           # HTTP / WS 入口与进程生命周期
packages/domain/     # 无框架领域对象与状态机
packages/application/ # commands / queries / orchestration
packages/contracts/  # OpenAPI、JSON Schema、事件类型
packages/persistence/ # SQLite、迁移、outbox
packages/workspace/   # 文件系统端口与安全实现
packages/adapter-kit/ # AgentAdapter SDK、测试夹具
adapters/{hermes,claude,codex,dsh}/
packages/testing/     # fake adapter、协议一致性测试
```

依赖方向： `adapters -> adapter-kit -> domain`；`core -> application -> domain`。Domain 不导入 HTTP、SQLite、Electron 或任何供应商 SDK。

跨模块规则

- 只通过命令、查询、领域事件和显式 Port 交互；禁止跨模块直接写表。
- 每个状态转换由单一模块拥有；例如 Run 状态只能由 runs 模块推进。
- 共享类型放 contracts，但共享业务逻辑仍归拥有模块，避免“公共包”变成垃圾场。
- 每次 Schema 修改必须包含迁移、向后兼容说明和契约测试更新。

单聊隐式命中一个 Agent，群聊只响应明确目标

单聊规则

- Conversation 必须绑定一个 agent_id。
- 每条 human 消息隐式生成一个 Invocation。
- Agent 忙时进入 FIFO；用户可取消队列项。
- 切换引擎实例创建新的绑定版本，不改历史消息。

群聊规则

- 只路由结构化 mentions。
- @all 仅群主、管理员或 PM 可用。
- 多个目标各建 Run，可按 group concurrency 限流。
- 未 @ 的消息只进入共享上下文，不自动触发执行。

路由算法

```
route(message, conversation):
    assert membership(message.sender)
    if conversation.kind == "direct":
        return [boundAgent(conversation)]
    targets = resolveMentionIds(message.mentions)
    targets = filterActiveMembers(targets)
    targets = policy.authorize(sender, targets, "invoke")
    return stableUnique(targets)
```

消息总线语义

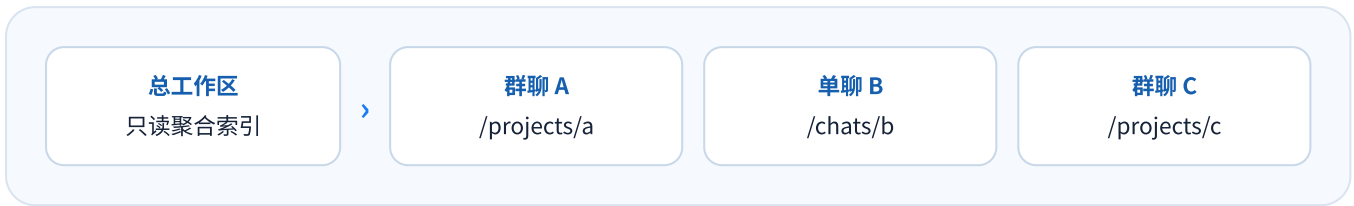
保证	实现	客户端含义
至少一次事件发布	事务 Outbox + event_id	按 event_id 去重
会话内有序	conversation sequence	检测缺口后拉取补偿
命令幂等	Idempotency-Key + scope	网络重试不重复发消息
最终消息唯一	run_id 唯一约束	delta 可重放，final 不重复

并发与公平

- **默认**：每个 Agent Instance 同一时间一个 active Run；不同实例互不阻塞。
- **群限额**：每个群默认最多 4 个并发 Run，超出进入可见队列。
- **优先级**：用户直接 @ 高于 PM 后台分派；恢复任务低于新消息。
- **背压**：WebSocket 只保留有限 delta；慢客户端转为“跳过增量，等待 final”。

回复约束：Agent 输出归属原会话，并引用触发消息 `replyTo`。Agent 不能仅凭文本声称身份；sender 始终由 Core 根据适配器连接确定。

每个聊天有独立工作区，总工作区只是聚合视图



会话状态机



工作区规则

规则	后端要求
路径归一化	解析真实路径后确认仍位于 workspace root；拒绝符号链接逃逸。
写入并发	请求带 expectedVersion；不匹配返回 409 与当前版本。
文件事件	合并短时间抖动，发出 workspace.changed；不传完整文件内容。
大文件	元数据与内容分开；内容支持 Range，预览设置上限。
删除	默认进入工作区回收区；永久删除必须单独确认。
Agent 访问	Run 建立时注入 workspaceId，不向适配器暴露任意宿主根目录。

上下文组装顺序

1. 系统策略与审批规则。
2. Agent 人格、能力与模型设置。
3. 群公告或单聊约束。
4. 最近有效 Summary；其后是摘要覆盖范围之后的原始消息。
5. 触发消息、附件引用与当前工作区元数据。

不要把工作区路径当身份：同一路径可以被重新绑定，但 workspace_id 永远不变；所有审计记录写 ID 与当时解析出的规范路径。

生命周期钩子



PM 提出团队方案，用户确认后才创建成员

编排状态机



TeamPlan 是可审阅对象

```
{
  "goal": "完成 OQQQ 后端 MVP",
  "roles": [
    { "role": "backend", "enginePreference": ["claude-code", "codex"],
      "skills": ["typescript", "sqlite"], "count": 1 },
    { "role": "reviewer", "enginePreference": ["hermes"],
      "skills": ["security-review"], "count": 1 }
  ],
  "tasks": [ { "id": "T1", "ownerRole": "backend", "dependsOn": [] } ],
  "requiresUserConfirmation": true
}
```

PM 可以做

- 从已安装 Agent 中推荐角色。
- 发现缺失能力并给出安装建议。
- 创建任务、依赖与验收条件。
- 在群中 @成员并汇总进度。
- 请求用户批准新增 Agent 或高风险能力。

PM 不可以做

- 静默安装、更新或卸载 Agent。
- 绕过写文件、命令与外部发送审批。
- 把一个会话的凭据复制到另一个会话。
- 自行扩大工作区根目录。
- 把“完成”建立在 Agent 自述而非验收证据上。

任务分派

任务状态	进入条件	可执行动作	退出条件
planned	TeamPlan 已确认	修改负责人/依赖	依赖满足
ready	无阻塞依赖	启动 Run	已认领
running	Run active	暂停、取消、追问	结果或失败
review	产物已提交	验收、退回	通过或重做
done	验收通过	纳入归档	项目关闭

MVP 简化：首版只实现一个 PM、显式确认、静态依赖图和手动验收；自动扩缩团队、Agent 竞标与成本优化放到后续。

摘要可重算，档案可移交，原始记录不可被覆盖

触发策略

默认每 30 条可计入消息触发检查；同时按上下文 token 水位、项目阶段切换和手动请求触发。阈值来自配置，不写死在前端。

覆盖策略

Summary 记录 `fromSequence`、`toSequence`、生成模型与源消息哈希。新摘要可以替代旧摘要进入上下文，但旧记录仍保留。

摘要结构

区块	内容	禁止
目标	当前目标与验收标准	未经确认的新目标
决定	已确认方案、接口与约束	把建议写成决定
进度	完成、进行中、阻塞	只写“已处理”
产物	相对路径、版本、校验摘要	嵌入凭据或绝对私密路径
待办	负责人、下一步、依赖	无归属的模糊事项

项目档案 Manifest

```
{
  "schemaVersion": 1,
  "project": {"id": "...", "title": "...", "status": "completed"},
  "conversation": {"id": "...", "lastSequence": 1842},
  "team": [{"agentId": "...", "engineId": "...", "role": "backend"}],
  "summaries": [{"id": "...", "range": [1, 1800], "sha256": "..."}],
  "artifacts": [{"path": "docs/api.md", "sha256": "..."}],
  "requirements": {"engines": [], "capabilities": [], "credentialKinds": []}
}
```

归档流程

1. 阻止新 Run，等待或取消活动 Run。
2. 生成最终摘要和任务验收报告。
3. 扫描凭据、临时文件和不允许的绝对路径。
4. 生成 manifest 与文件哈希；压缩包可选。
5. 写 Archive 记录，切换会话为 archived。
6. 恢复时先生成 dry-run 计划：缺失引擎、能力、凭据类型与路径冲突。

档案不包含：API Key、Token、Cookie、环境变量原值、供应商登录状态、系统级配置。
只记录恢复所需的“凭据类型”和配置占位。

状态、历史与运行证据分开存储

实体	关键字段	约束	索引
conversations	id, kind, title, state, workspace_id	kind: direct/group	updated_at
members	conversation_id, actor_id, role	复合唯一	actor_id
messages	id, conversation_id, seq, sender_id, kind, blocks	会话内 seq 唯一	(conversation_id, seq)
mentions	message_id, target_actor_id	结构化目标	target_actor_id
runs	id, trigger_message_id, agent_id, state	终态不可逆	conversation_id,state
run_events	run_id, seq, type, payload	run 内 seq 唯一	(run_id,seq)
approvals	id, run_id, action, state, expires_at	一次性决定	state,expires_at
engines	id, adapter_kind, version, health	adapter kind 唯一	health
agents	id, engine_id, config_ref, status	不存密钥原值	engine_id,status
capabilities	id, kind, source, version, manifest	kind 联合类型	kind,source
summaries	conversation_id, range, source_hash, body	范围不可重叠生效	conversation_id,to_seq
archives	id, project_id, manifest_path, sha256	不可变版本	project_id,created_at
outbox_events	event_id, aggregate_id, seq, payload	发布后标记	published_at

公共枚举

```

type RunState =
  | 'queued' | 'starting' | 'running' | 'waiting_approval'
  | 'cancelling' | 'completed' | 'failed'
  | 'cancelled' | 'cancelled_forced' | 'interrupted';

type MessageKind =
  | 'human' | 'agent' | 'summary' | 'system'
  | 'tool_event' | 'task_event' | 'file_event';

```

SQLite 约定

- 启用 WAL、foreign_keys 与 busy_timeout；迁移使用单调版本号。
- JSON payload 必须通过 JSON Schema 校验；常用查询字段拆列，不把所有数据塞进 JSON。
- 时间统一 RFC 3339 UTC；对用户显示时由客户端转换。
- 软删除记录 `deleted_at`；清理任务按保留策略物理删除。
- 消息与 outbox 同事务提交；发布成功仅更新 outbox 状态。

REST 负责命令与查询，统一返回可诊断错误

协议约定

基础

前缀 `/v1`；JSON 使用 camelCase；时间为 RFC 3339；列表使用 cursor；删除默认软删除。

幂等

创建消息、启动安装、创建 Run、归档等命令必须接受 `Idempotency-Key`。

并发

可变资源返回 `version`；更新携带 `If-Match` 或 `body.expectedVersion`。

错误

错误体包含稳定 code、用户可读 message、可选 details 与 traceId，不泄漏秘密。

```
{
  "error": {
    "code": "WORKSPACE_VERSION_CONFLICT",
    "message": "文件已被其他操作修改",
    "details": {"currentVersion": "sha256:..."},
    "traceId": "trc_..."
  }
}
```

会话与消息

方法与路径	用途	关键输入	响应
GET /conversations	会话列表	kind,cursor,limit	items,nextCursor
POST /conversations	创建聊天/群聊	kind,title,memberIds,workspace	Conversation 201
GET /conversations/{id}	会话详情	-	Conversation
PATCH /conversations/{id}	标题、公告、工作区	patch,expectedVersion	Conversation
GET /conversations/{id}/messages	分页消息	beforeSeq,limit	items,nextCursor
POST /conversations/{id}/messages	发送消息	clientMessageId,blocks,mentions	MessageReceipt 202
POST /messages/{id}/retry	重试对应 Run	agentId?	Run 202
POST /runs/{id}/cancel	取消运行	reason?	Run 202

消息请求示例

```
POST /v1/conversations/cv_01/messages
Idempotency-Key: 7f8d...
{"clientMessageId":"local_17","blocks":[{"type":"text","text":"请检查 API"}],
 "mentions":[{"actorId":"agt_reviewer","start":0,"length":4}]}
```

联系人、能力、工作区与项目共用同一版本规则

Agent 与联系人

方法与路径	用途	关键响应/动作
GET /engines	安装目录	版本、安装态、健康、configSchema、actions
POST /engines/{id}/install	安装引擎	operationId 202；进度走 WS
POST /engines/{id}/update	更新引擎	目标版本可选
DELETE /engines/{id}	卸载引擎	检查是否仍被 Agent 使用
GET /agents	好友列表	状态、模型、能力、分组
POST /agents	创建 Agent 实例	engineId、displayName、config
PATCH /agents/{id}	设置与分组	JSON Merge Patch
POST /agents/{id}/health-check	即时诊断	HealthReport

能力与第三方市场

方法与路径	用途	备注
GET /capabilities	已安装技能/连接器/插件	kind、compatibility、bindings
GET /marketplace/plugins	代理查询 dsh 市场	前端不直连，不保存市场凭据
POST /capabilities/install	安装能力	source、installSpec、expectedPermissions
POST /agents/{id}/capabilities/{cid}	绑定能力	校验引擎兼容性
DELETE /agents/{id}/capabilities/{cid}	解除绑定	不卸载共享能力

工作区与项目

方法与路径	用途	关键字段
GET /workspaces	总工作区索引	conversation、root、health
GET /workspaces/{id}/tree	目录树	path,cursor,depth
GET /workspaces/{id}/files/content	读取内容	path,range
PUT /workspaces/{id}/files/content	版本化写入	path,content,expectedVersion
POST /projects/{id}/team-plans	生成组队方案	goal,constraints
POST /team-plans/{id}/confirm	确认并建队	expectedVersion
POST /projects/{id}/archive	开始归档	includeWorkspace,format
POST /archives/{id}/restore-plan	恢复预检	targetWorkspace

一个版本化事件信封承载所有实时变化

连接与恢复

```
GET /v1/events?after=evt_0001842
Sec-WebSocket-Protocol: oqqq.v1

{
  "eventId": "evt_0001843", "sequence": 1843,
  "type": "message.delta", "occurredAt": "2026-10-07T14:20:31Z",
  "scope": {"conversationId": "cv_01", "runId": "run_08"},
  "payload": {"messageId": "msg_92", "delta": "正在检查"}
}
```

握手

连接后服务端发送 `system.ready`，包含 `protocolVersion`、`heartbeatMs` 与 `currentSequence`。

恢复

客户端提交最后确认的 `sequence`。若超出保留窗，服务端发 `system.resync_required`，客户端重拉 REST 快照。

事件目录

事件	关键 payload	前端动作
conversation.created/updated	conversation	更新会话列表与详情
message.appended	message	插入正式消息、替换本地回执
message.delta	messageId,delta,index	更新流式缓冲
message.completed	messageId,runId	清理流式态
run.state_changed	runId,state,reason?	更新停止/重试按钮
tool.approval_requested	approval,action,target	展示审批卡
tool.approval_resolved	approvalId,decision	锁定卡片
workspace.changed	workspaceId,paths,changeKind	失效目录缓存
agent.status_changed	agentId,status,health	联系人状态
engine.install_progress	operationId,phase,percent	安装进度
capability.install_progress	operationId,phase,percent	能力进度
project.archive_progress	archiveId,phase,percent	归档对话框

心跳与背压

- 服务端定时 `ping`，客户端回 `pong` 并附最后处理 `sequence`。
- `delta` 可被合并；`append`、`completed`、`approval` 与 `state_changed` 不可丢。
- 单连接待发送缓冲超过阈值时，先丢 `delta`，再发 `system.degraded`；不能悄悄丢正式事件。

适配器暴露能力与事件，不暴露供应商细节

```
interface AgentAdapter {
  readonly descriptor: EngineDescriptor;
  probe(ctx: ProbeContext): Promise<HealthReport>;
  install(req: InstallRequest, emit: OperationSink): Promise<InstallResult>;
  update(req: UpdateRequest, emit: OperationSink): Promise<InstallResult>;
  uninstall(req: UninstallRequest): Promise<void>;
  createSession(req: CreateSessionRequest): Promise<AdapterSession>;
  resumeSession(ref: SessionRef): Promise<AdapterSession>;
}

interface AdapterSession {
  run(input: AgentInput, signal: AbortSignal): AsyncIterable<AdapterEvent>;
  answerApproval(id: string, decision: ApprovalDecision): Promise<void>;
  cancel(reason?: string): Promise<void>;
  close(): Promise<void>;
}
```

标准事件联合类型

```
type AdapterEvent =
  | {type: 'assistant.delta'; text:string; index:number}
  | {type: 'assistant.completed'; blocks:ContentBlock[]}
  | {type: 'tool.requested'; call:ToolCall}
  | {type: 'tool.progress'; callId:string; detail:string}
  | {type: 'tool.completed'; callId:string; result:ToolResult}
  | {type: 'approval.requested'; request:ApprovalRequest}
  | {type: 'usage'; inputTokens?:number; outputTokens?:number; costUsd?:number}
  | {type: 'status'; state:'running' | 'idle'}
  | {type: 'error'; code:string; message:string; retryable:boolean};
```

能力描述必须来自探针

字段	示例	用途
sessionModes	one-shot, resumable, persistent	决定会话恢复方式
streaming	delta, final-only	决定 UI 是否显示流式
approvals	native, host-mediated, none	安全策略与可用功能
tools	native, mcp, dsh-bridge	能力绑定路径
structuredOutput	json-schema, jsonl, none	事件解析策略
interrupt	graceful, process-kill	取消语义

版本隔离：descriptor 同时记录 adapterVersion、engineVersion 与 protocolVersion。升级探针失败时保留旧版本，不自动把不兼容事件送入 Core。

一致性测试决定一个引擎能否进入稳定通道

适配器一致性测试

生命周期

安装、探针、启动、空闲、关闭、升级与卸载。所有外部进程必须可回收。

消息

流式顺序、唯一 final、结构化内容、错误、超时与异常退出。

会话

创建、继续、恢复与中断。若引擎不支持，必须返回明确能力错误。

工具与审批

请求、暂停、批准、拒绝、结果和取消；没有审批通道时不伪装支持。

黄金夹具

```
fixtures/adapters/contract/
happy-path.jsonl
approval-allow.jsonl
approval-deny.jsonl
cancel-mid-stream.jsonl
malformed-event.jsonl
process-crash.jsonl
resume-session.jsonl
```

兼容等级

等级	条件	产品呈现
stable	目标 OS 实测；全部必需夹具通过	默认可安装
beta	主链路通过；部分可选能力缺失	显示限制与版本范围
experimental	仅 one-shot 或接口快速演进	需用户显式开启
blocked	协议不匹配或安全边界不足	禁止启动

升级规则：引擎版本变化后重新 probe 并运行最小夹具；失败则自动回退到上一已验证适配器版本，不能静默继续。

日志与诊断

夹具只保存脱敏事件形状，不保存真实 prompt、凭据或文件内容。故障包包含版本、平台、退出码、事件类型序列和最后一个安全错误摘要。

四种引擎采用不同主路径，但输出同一协议

引擎	首选接入	降级	重点风险
Hermes	headless backend / JSON-RPC+WS（先做版本探针）	非交互 CLI	接口仍演进；需隔离配置与状态目录
Claude Code	Agent SDK（TS/Python）	<code>claude -p</code> + stream-json	权限模式、会话恢复、输出版本
Codex	Codex SDK / app-server	<code>codex exec</code>	sandbox 与 approval 组合、事件兼容
dsh	SDK profile + stdio JSON-RPC	headless one-shot	开发预览、SDK 协议与插件兼容

Hermes

将 Hermes 视为完整 Agent 引擎：保留其人格、记忆、技能和会话语义。适配器启动独立 profile，探测 headless 服务可用性，再把消息、工具与状态映射为 AdapterEvent。若当前安装缺少稳定服务接口，MVP 只开放 one-shot，并在 UI 标记“不可恢复”。

Claude Code

优先使用 Agent SDK 的类型化消息、工具审批回调和会话对象。CLI 回退使用 `-p` 非交互模式与结构化流输出；进程 stdout 只交给供应商解析器，stderr 进入脱敏诊断。0QQQ 的审批策略不能被 `allowedTools` 等参数弱化。

Codex

优先 SDK 或 app-server 以获得线程与结构化事件；CLI 回退使用 `codex exec`。启动前将 0QQQ workspace 映射为唯一可写根，并把 sandbox/approval 配置记录到 Run snapshot。禁止使用关闭沙箱的便捷模式作为默认。

dsh

dsh 同时可作为 Agent 引擎和能力底座。作为 Agent 时经 DshAgentAdapter 进入统一会话；作为能力底座时由 Capability Broker 管理插件、技能与连接器。两种角色共享安装但不共享未授权的会话上下文。

上线门槛：每个适配器必须在目标 OS 的实际已安装二进制上通过 probe 与协议夹具，不能只根据文档假设参数存在。探针结果缓存短时间，并在版本变化后强制重跑。

共同退出码映射

供应商情况	0QQQ code	retryable	用户动作
未安装/找不到二进制	ENGINE_NOT_FOUND	否	进入联系人安装页
未登录/缺凭据	ENGINE_AUTH_REQUIRED	否	打开安全配置流
限流	ENGINE_RATE_LIMITED	是	按 retryAfter 等待
协议无法解析	ENGINE_PROTOCOL_MISMATCH	否	回滚或更新适配器

MVP 以 SDK profile 驱动，headless 只作降级验证

首选：stdio JSON-RPC sidecar



图 2：每个 sidecar 使用独立 DSH_HOME/profile；Core 管理生命周期和安全边界。

启动协议

1. 解析固定版本的 dsh 可执行文件，建立最小环境变量白名单。
2. 以 SDK 或 SDK-minimal profile 启动子进程，cwd 指向会话工作区。
3. 发送 initialize；校验 serverInfo、protocolVersion 与能力。
4. 创建或恢复 session；把 sessionId 映射到 OQQQ adapter_session_ref。
5. 订阅 session 状态与事件；转换为 AdapterEvent。
6. 空闲超时后优雅 shutdown；超时再终止进程树。

事件映射

dsh 事件族	OQQQ AdapterEvent	处理
assistant chunk/message	assistant.delta/completed	去重、序号检查
tool call/result	tool.requested/completed	隐藏超长原始输出
approval asked/decided	approval.requested + resolve	若当前版本支持；否则禁用该动作
session status	status	驱动 Run 状态
subagent lifecycle	tool.progress / task_event	保留 parent-child 关联

降级：headless one-shot

```
dsh --profile headless "<task>"
# 仅用于：安装验收、健康检查、无会话恢复要求的简单任务
# 不假设 JSONL、交互审批或 resume；以当前二进制探针为准
```

当前边界：dsh 属于快速演进的软件。正式编码前必须用锁定版本实测 SDK profile、会话复用、取消、审批与事件字段；本文将它们定义为 OQQQ 的目标适配契约，而不是对任意 dsh 版本的兼容承诺。

插件市场

第三方插件广场由后端代理目录查询和安装。安装前解析 manifest、权限、兼容版本与来源；下载、校验、解包、启用分成独立阶段，每一步都通过 `capability.install_progress` 可见。

安全控制在 Core 收口，供应商设置只能更严格

四类敏感动作

动作	默认策略	审批卡必须显示
写入/删除文件	工作区内写需审批；读取按会话授权	路径、操作、差异摘要、发起 Agent
执行命令	每次审批或策略白名单	完整命令、cwd、网络需求
外部发送	每次审批	目的地、收件方、将发送的数据
凭据使用	安全存储引用；不向日志暴露	凭据类型、目标服务、用途

隔离

- 每个 Run 获得最小环境变量集合。
- 工作区采用规范路径与允许根校验。
- sidecar 使用独立进程组，取消可终止子进程树。
- 第三方插件默认无网络、无跨工作区访问。

审计

- 记录谁在何时请求何动作、用户决定与结果。
- 日志只保存摘要和哈希，不保存凭据与完整敏感内容。
- 审计事件只追加；管理员导出也需显式操作。
- traceId 贯穿 HTTP、Run、Adapter 与工具执行。

可靠性模式

故障	检测	处置	用户可见状态
Core 重启	启动扫描非终态 Run	标 interrupted；可安全重试	“运行已中断”
适配器退出	子进程 exit	终止会话；限次重启	Agent error
WebSocket 断线	心跳超时	指数退避；sequence 追赶	离线条
数据库忙	busy timeout	短退避；命令幂等重试	发送中
文件冲突	expectedVersion 不符	拒绝覆盖；返回当前版本	冲突处理
事件积压	consumer lag	合并 delta；保留正式事件	降级提示

配置与凭据

普通配置存数据库并版本化；凭据只存系统安全存储，数据库保存 opaque credentialRef。适配器只在启动或单次调用时获得最小引用解析结果，严禁写入命令行参数、URL、错误堆栈或档案。

默认拒绝：若引擎不能把工具请求交回 OQQQ 审批层，则该适配模式只能用于不具备敏感工具的只读任务，不能通过“全自动”参数绕过。

协议夹具证明兼容，端到端测试证明能用

测试金字塔

领域单测

状态机、路由、权限、摘要范围、归档清单。纯函数优先，不启动外部进程。

契约测试

OpenAPI、WebSocket schema、Adapter Kit。每个适配器重放同一黄金事件集。

端到端

真实 SQLite、文件系统和至少一个锁定引擎：发送、流式、审批、取消、恢复、归档。

MVP 必测场景

编号	场景	通过条件
E01	单聊发送与流式	唯一 final；刷新后历史一致
E02	群聊未 @	不创建 Run
E03	群聊 @ 两个 Agent	两个独立 Run；回复关联原消息
E04	写文件审批拒绝	文件未变；Run 明确结束或继续
E05	WebSocket 中断恢复	无正式事件丢失或重复
E06	客户端重复 POST	只产生一条消息与一组 Run
E07	适配器崩溃	Core 存活；Run 标 failed/interrupted
E08	文件 CAS 冲突	409；原文件不被覆盖
E09	30 条摘要触发	范围、哈希、原消息保留
E10	项目归档与 dry-run 恢复	缺项清单准确；无凭据泄漏

关键指标

API：请求量、p50/p95/p99、错误码、SQLite busy。

Bus：outbox lag、WS 连接、sequence gap、丢弃 delta 数。

Run：排队、首 token、总时长、取消、审批等待。

Adapter：probe 成功率、启动时长、协议错误、重启次数。

Workspace：读写量、冲突率、watcher 延迟。

Project：任务吞吐、返工、摘要耗时、归档失败。

日志字段： `traceId`、`conversationId`、`runId`、`agentId`、`adapterKind`、`eventType`、`durationMs`、`errorCode`。默认不记录消息正文。

发布门禁

- 类型检查、单测、迁移测试、OpenAPI breaking-change 检查。
- 四个适配器至少完成 probe；未完成者明确标为 experimental。
- Windows 与 Linux 路径、进程终止和编码测试。
- 依赖清单与第三方插件安装链路的来源校验。

先锁契约与真实引擎，再并行推进前后端

阶段计划

阶段	交付	退出门槛
P0 契约冻结	OpenAPI、WS Schema、Adapter Kit、fake adapter	前端 Mock 全闭环；契约测试通过
P1 运行时验证	dsh SDK/headless 实测；四引擎 probe	记录版本与能力矩阵；选择稳定主路径
P2 核心 MVP	SQLite、会话、消息、@路由、Run、WS	E01-E08 通过；断线可恢复
P3 联系人与能力	安装/更新/卸载、健康、技能/连接器/插件	联系人页真实联调；审批可见
P4 项目协作	PM 组队、任务、摘要、完结与归档	E09-E10 通过；档案无凭据
P5 生产强化	监控、升级回滚、备份、性能与安全审计	发布门禁、恢复演练、兼容矩阵

首批工程任务

仓库基线

- Monorepo 与 package boundaries
- contracts 版本策略
- SQLite 迁移框架
- CI 与 fix-report 约定

验证 Spikes

- dsh SDK profile 流式与取消
- Claude SDK 审批回调
- Codex SDK/app-server 会话
- Hermes headless 探针

MVP 验收清单

- 消息、联系人、工作区、技能、连接器、第三方插件广场与设置均有真实 API，不保留装饰按钮。
- 添加 Agent 后进入好友列表；点击 Agent 返回详情、健康、模型、能力和操作。
- 群聊只触发被 @ 的 Agent；单聊无需 @；执行状态、审批与取消实时可见。
- 每个群聊和单聊独立绑定工作区；总工作区只聚合，不改变隔离边界。
- 任一适配器退出不拖垮 Core；客户端断线后能按 sequence 恢复。
- 项目可生成摘要、归档并做恢复预检；任何凭据都不进入归档。

实施前必须确认

四项实测：锁定的 dsh 版本及 SDK 协议；Hermes 可用的稳定 headless 入口；Claude/Codex 在 Windows 的审批与取消行为；插件市场真实 API 的字段、限流和安装规范。未实测前不得把推测写进生产适配器。

实现以锁定版本实测与可执行契约为准

以下资料用于确认程序化入口、沙箱概念与第三方插件来源。供应商接口会变化，开发时应固定版本并保存 probe 结果。

参考资料

1. <https://code.claude.com/docs/en/headless>
2. <https://developers.openai.com/codex/cli/features>
3. <https://developers.openai.com/codex/concepts/sandboxing>
4. Hermes headless backend 变更记录
5. dsh CLI profiles 参考
6. <https://github.com/imsai-sh/dsh-1024store>
7. <https://github.com/imsai-sh/awesome-deepseek-harness-plugins>

内部基线：配套《OQQQ 前端开发对接文档》v0.3 规定 7 个一级入口、联系人主从布局、前端数据模型与 UI 状态；本文件的接口字段与事件名称按该基线对齐。